

Lecture 02: Linear Regression Review

Machine Learning Tutorial

School of Engineering, Westlake University

Fall 2026



Course announcements

Machine Learning Tutorial | September 10, 2026

Lab 01 (graded)

Due Sep 10, 2026 at 23:59

Submit your PDF to Canvas.

Lab 02 (practice)

Release: Sep 10, 2026

No submission required; no deadline.

Project 01

Release: Sep 10, 2026

Due Oct 22, 2026 at 23:59

Submit a PDF report and code.zip to Canvas.

All times are Beijing time (UTC+8).

Four views of linear regression

- Data: How do the parameters determine predictions and residuals?
- Objective: Which errors do MSE, MAE, and Huber emphasize?
- Optimization: What do gradients, learning rates, and batch sizes control?
- Geometry: Why do the normal equations describe a projection?

Review goal: explain the key ideas, their connections, and the conditions they require.

Regression and classification

Comparison	Regression	Classification
Output	Numerical response	Category label
Example	Predict weight from height	Detect a fraudulent transaction
Key distinction	Error magnitude has meaning	Label codes must not invent distances

- Encoding labels as 0/1 still requires a decision threshold and classification error costs.
- Least squares is a possible baseline, but its outputs need not lie in $[0, 1]$.
- Regression supports prediction and describes associations between inputs and outputs.

The linear model and notation

$$\hat{y}_i = w_0 + \sum_{j=1}^D w_j x_{ij} = \tilde{x}_i^T \tilde{w}$$

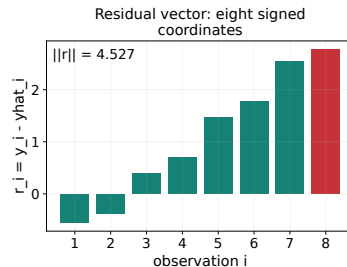
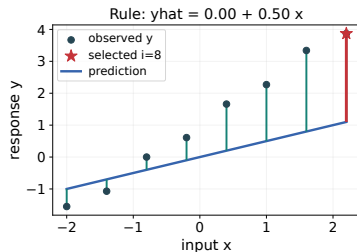
- Add a constant 1 to each input vector to include the intercept as a parameter.

$$p = D + 1, \quad X \in \mathbb{R}^{N \times p}, \quad w \in \mathbb{R}^p, \quad y, \hat{y} \in \mathbb{R}^N$$

$$\hat{y} = Xw, \quad r = y - Xw$$

We now omit the tildes. N is the sample size; p includes the intercept.

Residuals are signed prediction errors



$$r_i = y_i - \hat{y}_i, \quad r = (r_1, \dots, r_N)^T$$

- A positive residual means underprediction; a negative residual means overprediction.
- The intercept shifts all predictions equally. A slope change depends on the input.

From residuals to half-MSE

$$J(w) = \frac{1}{2N} \sum_{i=1}^N r_i^2 = \frac{1}{2N} \|y - Xw\|_2^2 = \frac{1}{2} \text{MSE}(w)$$

- Squaring prevents positive and negative errors from canceling.
- Large residuals receive a stronger penalty; dividing by N gives an average.
- The factor 1/2 preserves the minimizers and simplifies the gradient.

A zero sum of residuals can hide errors. Zero loss requires every residual to be zero.

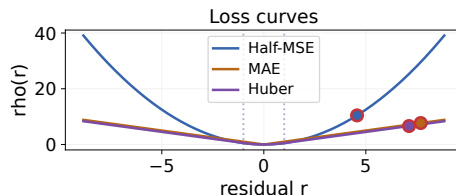
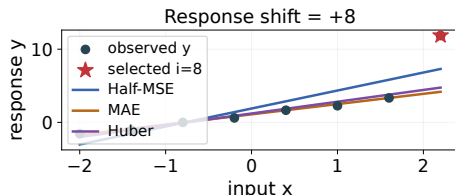
MSE, MAE, and Huber loss

Loss	Per-example loss $\rho(\mathbf{r})$	Residual derivative $\psi(\mathbf{r})$
Half-MSE	$r^2 / 2$	r : grows with the residual
MAE	$ r $	$\text{sign}(r)$; subgradient at 0
Huber	Quadratic, then linear	$\text{clip}(r, -\delta, \delta)$

$$\rho_{\delta}(r) = \begin{cases} \frac{1}{2}r^2, & |r| \leq \delta, \\ \delta(|r| - \frac{1}{2}\delta), & |r| > \delta. \end{cases}$$

Huber uses a threshold $\delta > 0$, chosen relative to the ordinary noise scale.

How an outlying response changes the fit



- Squared loss gives large residuals a larger gradient contribution.
- MAE and Huber bound residual influence; high-leverage inputs can still matter.

These losses use different scales. Their raw numerical values are not directly comparable.

What the gradient tells us

$$\nabla J(w) = -\frac{1}{N}X^T r = \frac{1}{N}X^T (Xw - y)$$

- Each component measures how the loss changes when one parameter changes.
- The gradient combines residuals, weighted by their input features.
- The negative gradient gives a local descent direction.

A zero gradient means the residual is orthogonal to every column of X ; it need not be zero.

Convexity and uniqueness

$$H = \nabla^2 J(w) = \frac{1}{N} X^T X$$

$$v^T H v = \frac{1}{N} \|Xv\|_2^2 \geq 0 \quad (\forall v \in \mathbb{R}^p)$$

- H is positive semidefinite, so the objective is a convex quadratic.
- Every local minimum is global. A zero gradient is sufficient for global optimality.
- Full column rank makes the objective strictly convex and the optimal parameters unique.

Gradient descent: direction and step size

$$w^{(t+1)} = w^{(t)} - \gamma \nabla J(w^{(t)}), \quad \gamma > 0$$

- Evaluate the gradient at the current parameters, then move in the opposite direction.
- The learning rate sets how far the parameters move at each update.
- A smaller step is more cautious; an oversized step can increase the loss.

Repeat the update to approach an optimum. Convexity alone does not make every step safe.

Learning rate and stability for the same objective

Use the same least-squares loss and gradient as before:

$$J(w) = \frac{1}{2N} \|Xw - y\|_2^2, \quad \nabla J(w) = \frac{1}{N} X^\top (Xw - y)$$

Let $w^\star$ be a minimizer, so its gradient is zero. Define the parameter error:

$$e^{(t)} = w^{(t)} - w^\star, \quad H = \frac{1}{N} X^\top X$$

Using the zero gradient at $w^\star$, the gradient and update become:

$$\begin{aligned} \nabla J(w^{(t)}) &= \frac{1}{N} X^\top [(Xw^{(t)} - y) - (Xw^\star - y)] = He^{(t)} \\ e^{(t+1)} &= e^{(t)} - \gamma He^{(t)} = (I - \gamma H)e^{(t)} \end{aligned}$$

Here e measures parameter error. The sample residual remains $r = y - Xw$.

Curvature and the learning-rate bound

Use an orthonormal eigenbasis of H : $Hv_j = \lambda_j v_j$, with $\lambda_j \geq 0$.

Project the same parameter error onto direction v_j :

$$e_j^{(t)} = v_j^\top e^{(t)}, \quad e_j^{(t+1)} = (1 - \gamma \lambda_j) e_j^{(t)}$$

For $\lambda_j > 0$, this error component shrinks exactly when:

$$|1 - \gamma \lambda_j| < 1 \iff -1 < 1 - \gamma \lambda_j < 1 \iff 0 < \gamma \lambda_j < 2$$

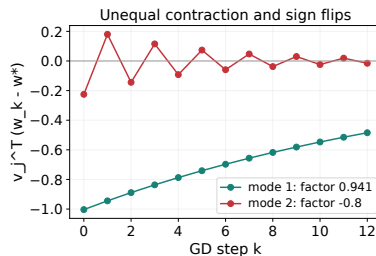
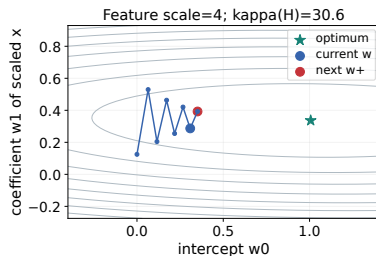
Every positive-curvature direction must satisfy this condition:

$$0 < \gamma < \min_{\lambda_j > 0} \frac{2}{\lambda_j} = \frac{2}{\lambda_{\max}(H)}$$

The steepest direction gives the tightest bound on the shared learning rate.

Zero-curvature components stay fixed and do not affect the training fit.

Ill-conditioning and zig-zag trajectories



- Unequal curvature causes zig-zags and slow progress along the valley.
- Changing feature units changes the Hessian and may require a new step size.

Tune the step size for stability. Use scaling or preconditioning for curvature.

A larger batch cannot fix ill-conditioning in full-gradient descent.

GD, SGD, and mini-batch SGD

$$g_i(w) = x_i(x_i^\top w - y_i), \quad g_{\mathcal{B}}(w) = \frac{1}{B} \sum_{i \in \mathcal{B}} g_i(w)$$

Method	Examples per step	Main trade-off
Full-batch GD	N	Exact gradient; more work per step
SGD	1	Cheap updates; greater noise
Mini-batch SGD	B, usually between 1 and N	Balances work and noise

- Compute every sample gradient in a batch at the same current parameter vector.
- At $B = N$, we recover the full gradient. Batch size and learning rate are different choices.

What an unbiased mini-batch gradient means

Fix the current parameters w and sample a batch uniformly without replacement.

$$\mathbb{E}_{\mathcal{B}}[g_{\mathcal{B}}(w)] = \nabla J(w)$$

- Across repeated draws at the same parameters, the average batch gradient is correct.
- A single sampled gradient can still point poorly and increase the full loss.
- Larger batches reduce sampling noise; using all N samples removes it.

Batch size controls gradient noise. Learning rate controls the size of the update.

Zero gradient and the normal equations

$$\nabla J(w^*) = 0 \quad \Longleftrightarrow \quad X^T(y - Xw^*) = 0$$

$$\boxed{X^T X w^* = X^T y}$$

- There are p scalar equations: the residual is orthogonal to every column of X .
- For this differentiable convex objective, the normal equations imply global optimality.
- The Gram matrix need not be invertible, and the optimal parameters need not be unique.

Least squares is the optimization problem. The normal equations characterize its solutions.

Least squares is an orthogonal projection

Every prediction is a linear combination of X 's columns:

$$\hat{y}^* = Xw^* \in \text{col}(X), \quad r^* = y - \hat{y}^*$$

The normal equations make the residual orthogonal to every vector in that space:

$$X^T r^* = 0 \implies (Xa)^T r^* = a^T X^T r^* = 0 \quad (\forall a \in \mathbb{R}^p)$$

This orthogonal decomposition identifies the projection:

$$y = \hat{y}^* + r^*, \quad r^* \perp \text{col}(X) \implies \boxed{\hat{y}^* = P_{\text{col}(X)} y}$$

For any other prediction $z \in \text{col}(X)$, orthogonality gives:

$$\|y - z\|_2^2 = \|r^*\|_2^2 + \|\hat{y}^* - z\|_2^2 \geq \|r^*\|_2^2$$

The projection is unique, even when X is rank deficient.

Unique parameters versus unique training predictions

Quantity	X: full column rank $\text{rank}(X) = p$	X: rank deficient $\text{rank}(X) < p$
Optimal parameters w^*	Unique	Infinitely many
Optimal training predictions Xw^*	Unique	Unique
Optimal residual $r^* = y - Xw^*$	Unique	Unique
Optimal loss $J(w^*) = \ r^*\ _2^2 / (2N)$	Unique	Unique

$$Xv = 0 \implies X(w^* + tv) = Xw^* \quad (t \in \mathbb{R})$$

- Moving along a null-space direction leaves training predictions and loss unchanged.
- Unique training predictions do not guarantee unique predictions on new inputs.

Closed-form expressions and numerical solvers

$$\text{rank}(X) = p \implies w^* = (X^T X)^{-1} X^T y$$

Design matrix	Mathematical result	Numerical consideration
Full rank; well conditioned	Unique parameters	Use a least-squares solver
Full rank; nearly collinear	Still unique	Sensitivity; inspect singular values
Rank deficient	Infinitely many minimizers	SVD: minimum-norm solution

In practice, prefer a least-squares routine based on QR or SVD.

Python: `w, residuals, rank, s = np.linalg.lstsq(X, y, rcond=None)`

Key ideas from four views

View	Key concepts	Main idea
Data	Regression / classification	Predict numerical values or class labels.
Objective	Half-MSE / MAE / Huber	Different ways to penalize prediction errors.
Optimization	Full-batch GD / SGD / mini-batch GD	Use all examples, one example, or a batch per update.
Geometry	Normal equations / orthogonal projection	Least squares projects y onto $\text{col}(X)$.

Next lecture and further reading

- Next: the probabilistic interpretation of linear regression and maximum likelihood (MLE).
- Next: overfitting, underfitting, polynomial regression, Ridge, and Lasso.

Source: Tao LIN, Lecture 2: Linear Regression from Four Views, Fall 2026.

Further reading from the lecture:

- Stanford CS229 Lecture Notes, Chapter 1.
- Bishop, Pattern Recognition and Machine Learning, Section 3.1.